

Everything But The Bucket

Cloudflare R2 needed no code: it is the same S3 API the adapter already speaks. What it needed was the eight answers you asked for before implementation, three new refusals so a misconfigured deployment cannot start, and a way to **prove** a bucket is private rather than promise it. All of that is done. **One thing is not, and cannot be from here: no Cloudflare account exists, so no real bucket has been touched.**

Branch **claude/create-app-repository-1cxsih**

HEAD **621c2e0**

Unit tests **683 passed**

Browser tests **46 passed**

Mutations killed **27 / 27**

Real R2 bucket **not yet**

Ledger touched **none**

READ THIS FIRST: WHAT IS BLOCKED, AND ON WHAT

The milestone asked for a real end-to-end test against an actual R2 bucket. **I cannot do that step.** It needs a Cloudflare account for AFRI NEXT TECHNOLOGIE, a bucket, and an API token — an account with a bill attached, created by you, with credentials that must not pass through me or through Git.

This environment does hold ambient AWS keys belonging to the sandbox itself. **I did not use them.** They are not Afrinext's, they are not R2, and creating storage on somebody else's account would be the wrong answer to the right question.

So everything below runs against the in-repo bucket fixture that verifies every signature. **Section H is the exact list of what I need from you**, and the one command that turns the whole verification list loose on the real bucket once it exists.

A · The eight answers	B · Secrets	C · Failing closed	D · Proving it is private	E · Render
F · The journey	G · Mutations & results	H · What I need from you		

A

The eight answers, before implementation

All eight live in `docs/providers/r2/README.md` , written against Cloudflare's current documentation rather than memory. In brief:

#	ASKED	ANSWER
1	The exact R2 architecture	Unchanged from the approved design. Uploads go browser → Afrinext → R2; downloads go R2 → Afrinext → browser. The browser never talks to R2, in either direction.
2	Every required variable	Seven. <code>CONTENT_STORAGE=s3</code> , endpoint, region (<code>auto</code> — R2 has no AWS regions), bucket, access key, secret key, path style.
3	Which are server-only secrets	Two: the access key and the secret key. Section B has what structurally keeps them that way.
4	Bucket naming and environment separation	One bucket per environment, never shared , each with its own token scoped to it alone. <code>-prod</code> , <code>-preview</code> . CI gets no bucket at all.
5	CORS is not opened unnecessarily	No CORS policy at all , which is R2's default. CORS governs requests a browser makes to the bucket, and no browser makes one.
6	Public bucket access disabled	R2 buckets are private by default. The <code>r2.dev</code> subdomain stays Disabled and no custom domain is connected.
7	An unauthenticated browser cannot reach it	Not asserted — tested . Section D.
8	Render gets secrets through Render	<code>sync: false</code> on all four sensitive values. Section E.

WHY R2, IN ONE LINE

Egress is free. A catalogue whose product *is* repeated downloads pays its bill in egress, and on R2 that line is zero. The adapter stays S3-compatible regardless, so the decision costs four variables to reverse.

B Two secrets, and what keeps them server-only

Only the access key and secret key are credentials. Three structural facts keep them out of a browser, and **not one of them is a convention somebody has to remember**:

1 No `NEXT_PUBLIC_` prefix.

Next.js inlines a variable into client JavaScript only if its name starts with that. None do, so the bundler cannot put them in a browser bundle even by mistake.

2 `packages/core` is never imported by a client component.

The adapter is built in a route handler and a server action, and the eslint rule that forbids Next.js inside `packages/*` keeps that boundary from eroding.

3

They are never logged.

A test captures the process's actual stdout during two failed operations and asserts the key, the secret, `AWS4-HMAC-SHA256` and `Credential=` are all absent from what was written.

The **endpoint and bucket name are not credentials** — useless without a token — but they stay out of pages and out of Git anyway, because naming your storage account is free reconnaissance. The browser test asserts neither appears in the library or product HTML, and it now reads the names from the *configured* bucket rather than the fixture's, so the assertion keeps meaning something on a real run.

c

Production fails closed, with no path back to a disk

CONFIGURATION	RESULT
<code>s3</code> , all five values, https endpoint	runs
any of the five missing or empty	throws at startup
endpoint is <code>http://</code> off loopback	NEW — throws at startup
endpoint is set but is not a URL	NEW — throws at startup
<code>filesystem</code> under a production build	throws — and <code>render.yaml</code> no longer carries the opt-out
a name that is neither	throws at startup

THE TWO NEW REFUSALS, AND WHY EACH IS WORTH A GUARD

A plaintext endpoint. SigV4 *signs* a request; it does not encrypt it. Over `http://` the `Authorization` header and the seller's files cross the network readable by anything on the path — and a typo that drops the `s` is not an error the adapter would otherwise notice, because it would *work*. Loopback is the only exception, on the literal addresses rather than the name `localhost` , which a hosts file can point anywhere.

An endpoint that is not a URL. This one was a live defect, found by a test I had written for something else. `CONTENT_S3_ENDPOINT=example.com` constructed cleanly, started the server, published the storefront, and would have failed *at the first*

buyer's download — the exact failure shape this whole configuration path exists to prevent. It is parsed in the constructor now.

D

"Public access is disabled" as a test, not a screenshot

A dashboard screenshot is a claim about a setting somebody made once, on a console other people can also reach. This is the same claim expressed as a request that must be refused:

```
pnpm --filter @afrinext/core verify:storage
```

Against whatever `CONTENT_S3_*` is in the environment it writes one probe object, reads it back, and then **fetches the object's URL with no `Authorization` header at all** — exactly what a browser that guessed the URL would send. R2 must refuse. It probes the bucket root the same way, so a bucket that lists its contents is caught too. Then it proves a wrong secret is rejected, so the refusals mean the bucket enforced something rather than the script failing to reach a server that would have said yes. Finally it deletes the probe.

A CONNECTION ERROR IS A FAILURE, NOT A PASS

The tempting shortcut — "the unsigned request didn't return 200, so we're private" — passes every misconfiguration that never reaches the provider at all. "Refused" and "could not connect" are different facts and the check reports them differently. Mutation **S8** conflates them and dies.

The logic lives in `packages/core/src/content/verify.ts` rather than in the script, for two reasons. It can be **pointed at a bucket that really is public** — the test bucket has a `publicReads` mode for exactly this — and watched to say so; *a safety check nothing has ever seen fail is one nobody knows the shape of*. And it returns structured results and formats nothing, so the CLI decides what reaches a terminal, which is what keeps a credential out of the output.

RUN AGAINST	VERDICT
the signature-verifying fixture	8 checks, all pass, exit 0
a bucket serving unsigned reads	"THE BUCKET IS PUBLIC — an unauthenticated GET returned 200", exit 1
an endpoint nothing answers	failure, never a pass

The middle row is the one that matters, and note *what still passed in it*: the write, the read-back, the checksum, the content type. **A round-trip test alone would have shipped that bucket.**

E

Render: four values it must ask you for

`render.yaml` now names R2 and carries **no credential**. The endpoint, bucket, access key and secret key are declared `sync: false` — Render's marker for "set in the dashboard, not in this file" — the same posture `DATABASE_URL` and `SESSION_SECRET` already have. A value pasted into that file would be committed, pushed, and readable in git history forever, including after the line is deleted.

TWO CONSEQUENCES YOU SHOULD AGREE TO BEFORE THE NEXT DEPLOY

1. The service will not start until you set them. That is the fail-closed behaviour working, not a regression — but it does mean the preview is down between this merge and your filling in those four fields.

2. The mounted disk is removed, and removing it deletes the volume. It existed only for filesystem storage. It also pinned the service to a single instance — Render will not run two of a service that has a disk — which would have made the multi-instance guarantee untestable on the very deployment meant to demonstrate it. The volume holds preview uploads only, and those do not survive the move to a bucket. **A production service with real files would migrate the objects first.**

F

The whole verification list, on whichever bucket is configured

The browser journey now takes `E2E_REAL_BUCKET=yes` plus the five values and drives the real provider instead of the fixture. **Same specs, same assertions, different bucket** — that is the point: a pass means the provider behaves the way the fixture claims providers behave. The spec prints which bucket it ran against, so a green run is never ambiguous.

It was extended to cover the list you gave, end to end, across two instances:

CHECKED	HOW
exact bytes	seller uploads on A, buyer downloads on B, body compared
content type preserved	<code>application/pdf</code> asserted on the response header
filename and disposition	<code>attachment; filename="Le fichier PDF"</code> , derived from the seller's title and stripped safe
version pinning	the seller publishes a real second version from instance A; the buyer is still served v1's bytes, and never v2's
a version not owned	requesting v2's asset is refused 403
download limit, second download	limit set to 2 through the seller's own control; the second succeeds
limit exhaustion	the third is 429 with <code>download_limit</code> — specific, because it is a fact about their own purchase
suspended store revokes	store suspended → 403; un-suspended → 200, so the refusal was demonstrably the store
another buyer cannot access	a third signed-in stranger is refused
storage key, bucket, endpoint absent from HTML	library and product pages fetched and searched, using the configured names
credentials never in logs or responses	stdout captured during two failed operations and searched
refund revokes access	domain and browser tests, unchanged and still green
storage failure does not consume a download	domain test with an injected 503 — not reproducible on a real bucket , since it needs the provider to fail on command

Every row above is currently green against the fixture. The last row is the only one that will stay fixture-only, and deliberately: making a real bucket fail on demand is not something to build.

G

Mutations and results

The matrix grew from 23 to 27. The four new ones all attack this milestone's guards:

#	THE LIE THE MUTATION TELLS	RESULT
S6	A plaintext endpoint is accepted anywhere, not only on loopback.	killed · 1 failing

#	THE LIE THE MUTATION TELLS	RESULT
S7	The public-access probe passes regardless of what the unsigned request returned.	killed · 2 failing
S8	A bucket that cannot be reached counts as a bucket that is private.	killed · 1 failing
S9	A malformed endpoint is discovered at the first download instead of at startup.	killed · 2 failing
S1–S5, D1–D18	The existing matrix, re-run in full against this HEAD.	all 23 killed

S8 SURVIVED THE FIRST RUN, AND THE BUG WAS IN MY TEST

"A bucket that cannot be reached counts as private" survived — not because the product was wrong, but because the test meant to cover it pointed at an endpoint so broken that the run failed at the *write* and never reached the probe. Its assertion, `result.ok === false`, was satisfied by that earlier failure. **It would have passed no matter what the probe did.**

The probe is now its own function, tested against all three answers a bucket can give. This is the second time in two milestones the matrix has found a weak test rather than a weak line of code, which is most of what it is for.

27 mutations, 27 killed, 0 survivors. Source verified pristine afterwards.

CHECK	RESULT
<code>pnpm lint</code>	clean
<code>pnpm typecheck</code>	clean
<code>pnpm test</code>	683 passed
<code>pnpm build</code>	compiled
<code>pnpm test:e2e</code>	46 passed
<code>verify:storage</code> against the fixture	8 / 8 checks
Mutations	27 / 27
Migrations	none — this milestone adds no schema
CI	3 / 3 green
Real R2 bucket	not tested — no account exists

No change to `ContentStorage` 's shape, the entitlement model, refunds, payments, the ledger, payouts, the admin console or the seller dashboard. **iPayMoney remains unimplemented and its sandbox pending.**

H What I need from you, exactly

Nine steps, and only the first six need a person. They are in `docs/providers/r2/README.md` too, so they do not live only in this PDF.

1 Create the Cloudflare account.

For AFRI NEXT TECHNOLOGIE, and enable R2 on it.

2 Create the production bucket.

Name it `afrinext-content-prod` . Pick a jurisdiction deliberately if data location matters — the endpoint changes if you do.

3 Confirm public access is off.

Under the bucket's Settings, *r2.dev subdomain* must read `Disabled` , and no custom domain may be connected. Both publish a bucket; only one of them uses the word.

4 Leave CORS unset.

There is nothing to configure. A browser never talks to the bucket, so a CORS policy on it would mean something upstream had changed.

5 Create the API token.

Permission *Object Read & Write*, scoped to that bucket only. Copy the Access Key ID and the Secret Access Key — Cloudflare shows the secret once and never again.

6 Repeat 2–5 for the preview bucket.

`afrinext-content-preview` , with its own token. A leaked preview credential must not be able to reach a real seller's file.

7 Set four values in Render.

Endpoint, bucket, access key, secret key, under the service's Environment. The other three come from `render.yaml`. Until this is done the service will not start.

8 Verify the bucket.

`pnpm --filter @afrinext/core verify:storage`. It must end `ALL CHECKS PASSED`.

9 Run the journey against it.

`E2E_REAL_BUCKET=yes pnpm test:e2e` — the whole list in section F, on the real bucket.

HOW TO GIVE ME THE CREDENTIALS, IF YOU WANT ME TO RUN 8 AND 9

Put them in the environment, not in a message and never in a file I commit — the same way `DATABASE_URL` already reaches this session. If you would rather run those two commands yourself, that is the safer choice and the output is short enough to paste back. **Either way I will not have written a credential into the repository.**

Next milestone, once the bucket exists: steps 7–9 above, then the deployment checks you listed — Render environment verified, deployed SHA confirmed, a real upload and a real download through the deployed instances, and the full suite re-run against the real bucket. Until then this milestone is honestly described as **configured and proved against a fixture, not against R2**.